Visitor Pattern — Advanced File System

STUDENT	GROUP	STACK
Zhakanov Sanzhar	SE-2526	Java 17 • Maven • JUnit 5

1. Purpose

Build a small in-memory hierarchical file system and use the behavioral Visitor pattern to perform three independent operations over the same object structure: compute total file size, search for matching files, and collect tree statistics. The model supports arbitrary directory nesting and rejects invalid structural changes.

2. Architecture and responsibilities

Component	Role in the implementation
Element / AbstractElement	Shared accept(Visitor) contract; name and parent are stored in the abstract base class.
FileNode	A file with a non-negative byte size; delegates accept to visit(FileNode).
DirectoryNode	An ordered list of children; addChild validates name uniqueness, ownership and acyclic nesting.
Visitor	Two visit overloads plus start(root) for initializing each operation.
FileSystemWalker	Owns depth-first traversal and dispatches every reachable element exactly once.
TotalSizeVisitor	Adds file sizes; directories contribute no bytes.
SearchVisitor	Applies a filename-substring filter and inclusive minimum size, then sorts root-relative paths.
StatisticsVisitor	Counts files/directories and measures the greatest depth from the selected root.

3. Traversal and double dispatch

The walker is the only traversal owner; visitors process the current node and never recurse through child collections. A run begins with visitor.start(root), resets accumulated state, and walks the selected subtree in depth-first order.

Two dispatch steps occur: (1) the concrete element executes accept(walker), selecting walker.visit(FileNode) or walker.visit(DirectoryNode); (2) the walker calls element.accept(realVisitor), selecting the corresponding overloaded visitor operation. No instanceof, reflection or type casts are needed.

4. Clean Code practices (five examples)

Practice	Evidence in code and justification
Meaningful naming	TotalSizeVisitor.getTotalSize() clearly expresses the value returned and avoids ambiguous names.
Single Responsibility	FileSystemWalker.walk() alone manages traversal; each visitor performs only its own calculation.
Small methods	FileNode.accept() and DirectoryNode.accept() delegate in one statement, keeping dispatch logic simple.
Encapsulation	DirectoryNode.getChildren() returns List.copyOf(children), preventing callers from modifying its internal collection.
No type-checking chains	Visitor.visit(FileNode) and visit(DirectoryNode) use overloading instead of instanceof branches.

5. Automated verification — JUnit 5

Nine test methods across seven test classes are defined for the eight required verification checkpoints. All tests are designed to run from the repository root with mvn clean test; they require no IntelliJ configuration or interactive input.

ID	Test class	Coverage
T1	VisitorDispatchTest	Correct overloaded visit methods
T2	TotalSizeVisitorTest	Size across multiple nested levels
T3–T4	SearchVisitorTest	Combined filters, sorted root-relative paths
T5	StatisticsVisitorTest	File count, directory count, max depth
T6	EmptyStructureTest	Empty root and no-match search
T7	RepeatedExecutionTest	No state leakage on repeat runs or root changes
T8	TreeIntegrityTest	Negative size, duplicate names, cycles, parents, immutable child view

Test run result: 9 JUnit test methods are present in the repository. Final Maven pass/fail totals must be verified in the student’s local terminal before submission; this report does not claim an unobserved run.

6. Demonstration and expected output

The demonstration tree contains four files: readme.txt (120 B), photo.png (900 B), documents/notes.txt (200 B), and documents/projects/report.txt (350 B). It contains three directories, including root. The search uses the substring .txt with a minimum size of 200 bytes.

```
Total size: 1570 bytes

Matching files:
documents/notes.txt
documents/projects/report.txt

Statistics:
Files: 4
Directories: 3
Maximum depth: 3
```

The total is 120 + 900 + 200 + 350 = 1570 bytes. Search paths exclude the chosen root name and use the / separator; results are sorted lexicographically. Root depth is 0; the deepest file, report.txt, is at depth 3.

7. Design choices, invariants and trade-offs

Object integrity. File sizes cannot be negative. addChild rejects duplicate sibling names, null children, reuse of an element owned by another parent, and attempts to create an ancestor cycle. The returned child list is not writable. Empty directories and a single-file root are valid structures.

Repeatability. Every operation initializes itself in start(root): counters become zero, search results are cleared, and a new root is recorded. Thus visitors can be reused without multiplying values across runs.

Extension boundary. Adding a new operation, such as statistics, requires implementing Visitor only; existing FileNode and DirectoryNode accept methods stay unchanged. Conversely, introducing a new element type would require adding another visit overload and updating every Visitor implementation. This is the main Visitor trade-off.

8. Repository and submission

Repository: https://github.com/lonelysun21/SDP_6_assignment

Reproduce: `mvn clean test` (test suite) and `mvn compile exec:java` (console demonstration). Submit the public GitHub URL together with this PDF to Moodle. The UML class diagram appears on the separate appendix page and is excluded from the three-page report limit.

References: AITU Software Design Patterns, Assignment 06 “Visitor Pattern — Advanced File System”; project source and tests in the linked repository.

Appendix A | UML Class Diagram

Not included in the three-page report limit

